

Unidad Temática N°1

~ Programación Orientada a Objetos ~

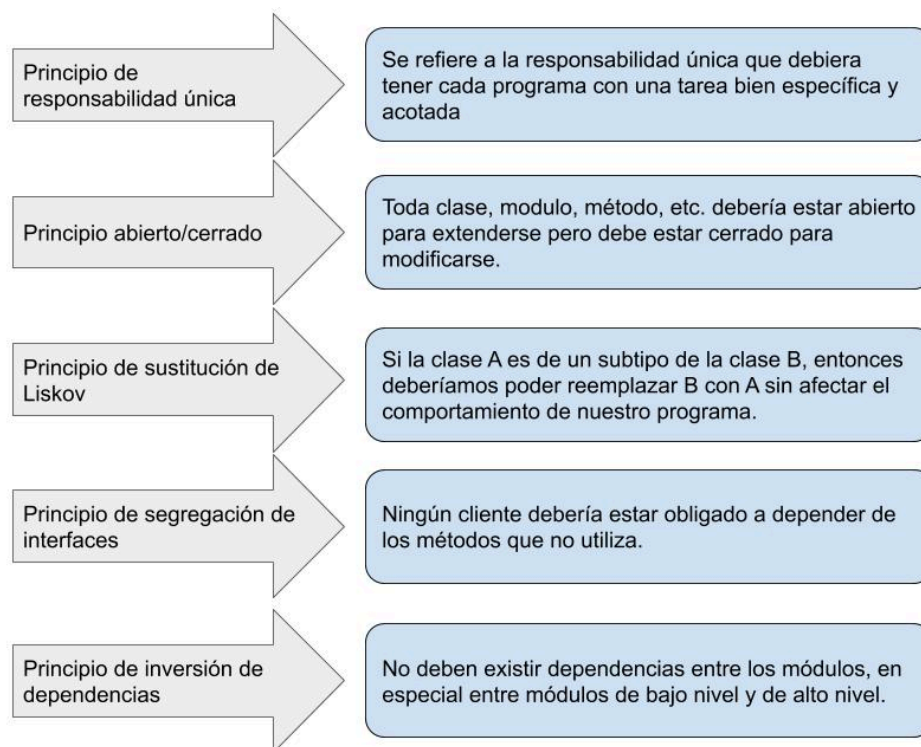
"La esencia del modelado es la abstracción."

~ Grady Booch

Programación Orientada a Objetos

La programación orientada a objetos es un paradigma que permite el abordaje de los problemas y búsqueda de soluciones mediante el establecimiento de un paralelismo con la realidad, agrupando las instrucciones en entidades modulares que reciben el nombre de **clase** y que permiten -a través del uso de diversas técnicas- organizar el código maximizando la reutilización de instrucciones, favoreciendo legibilidad y facilitando mantenibilidad.

Los principios básicos de la programación orientada a objetos se encuentran resumidos en el acrónimo mnemónico S.O.L.I.D. :



Fuente: <https://experto.dev/principios-solid-con-ejemplos/>

Clase

Generalidades

Una **clase** es un archivo de texto en el que se especifican, a nivel general, las características propias de un tipo particular de objeto; en un sentido más práctico se puede pensar a una clase como una plantilla en la que se definen determinados rasgos sin valores predefinidos.

Por ejemplo, se puede crear la clase Puerta definiendo atributos como altura, ancho, profundidad, color y material, lo que permite modelar tanto una puerta de madera como una de metal o de plástico, de cualquier color y de variadas dimensiones; sólo deben asignarse los valores de estos atributos al momento de crear una instancia.

Clase, Tipo, Objeto e Instancia

Aun cuando es bastante habitual intercambiar la palabra **clase** por **objeto**, poseen significados distintos: una **clase** es la especificación, es decir, los lineamientos que definen a un **tipo de objeto**; en cambio un **objeto** es una **instancia**, es decir, un espacio en la memoria RAM que se ha reservado para alojar los valores que el usuario quiera asignarle.

Por cada clase es posible crear tantos objetos como sean necesarios; cada uno requerirá espacio en la memoria RAM para albergar sus valores.

Cuerpo de una clase

Una clase posee un **nombre** que la identifica, un nivel de **visibilidad**, y opcionalmente palabras reservadas que indican si se **extiende** de otra clase o si, además, **implementa** una interfaz.

También presenta **atributos**, que son las variables a las que se les asignará valores, y **funciones**, que son las estructuras que delimitan bloques de instrucciones.

Genéricamente hablando, tanto los atributos como las funciones son conocidos como **miembros**.

Otro elemento importante es el **constructor**, que es el método que se invoca cuando se crea una instancia y el lugar en el que todas las asignaciones y acciones que deben

efectuarse al momento de crear un objeto, deben ocurrir; no es obligatorio explicitarlo si no se requiere realizar tarea alguna.

Visibilidad

Todos los miembros poseen **visibilidad**, que es un indicador sobre el **nivel de exposición**, tipo de dato al que referencia (en el caso de los atributos) o devuelve (en el caso de las funciones), y un **nombre** que los identifica.

Entender el concepto de visibilidad requiere pensar a las clases como elementos herméticos que impiden que sus miembros sean accedidos desde el exterior, imposibilitando que los objetos instanciados de una determinada clase puedan relacionarse con otros.

Para que los objetos puedan interactuar entre sí, es necesario explicitar qué miembros pueden ser vistos desde el exterior, y es allí en donde la visibilidad o nivel de exposición entra en juego.

- **Privado:** los miembros con este nivel de visibilidad pueden ser referenciados únicamente desde el interior de la clase y no presentan salida al exterior; las subclases no pueden acceder a ninguna función o atributo de su padre. Es el nivel más restrictivo.
- **Protegido:** es un poco más flexible que el Privado ya que permite que los miembros con este nivel de exposición sean referenciados por clases que se encuentran dentro del mismo paquete, como también por las subclases que extienden de la clase que presenta los miembros protegidos.
- **Público:** es el menos restrictivo de todos al permitir que cualquier clase pueda referenciar a los miembros con este nivel de visibilidad.
- **Sin modificador:** no se suele emplear mucho y es un intermedio entre el Privado y el Protegido, ya que únicamente permite el acceso a los miembros desde la misma clase o desde aquellas que se encuentran en el mismo paquete, excluyendo las subclases.

Modificador	Clase	Paquete	Subclase	Mundo
private	✓	✗	✗	✗
(no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Funciones, Setters y Getters

Como se mencionara anteriormente, las funciones son agrupadores de instrucciones que tienen como propósito realizar un conjunto de operaciones acotadas, **encapsulando** el comportamiento.

El concepto de **encapsulación** es muy importante para la programación orientada a objetos ya que permite trabajar conociendo qué hace una función pero no cómo, además de tener la posibilidad -si el código está debidamente estructurado- de reutilizarlo, es decir, invocarlo tantas veces como sea necesario sin tener que repetir código.

Por convención, salvo casos muy particulares, todo atributo debe ser privado y ser accedido mediante setters y getters, que no son más que funciones públicas que permiten la asignación o la lectura del valor de una determinada variable, respectivamente.

De este modo, es posible crear un atributo que sea accedido solamente para la lectura, únicamente para la escritura, o para ambos; de lo contrario, si contara con visibilidad pública, estarían disponibles las dos operaciones.

Ejemplo de de una clase

```
public class Person
{
    private string name;
    private string lastname;

    public Person()
    {
        this.name = "Juan";
        this.lastname = "Perez";
    }
}
```

```
}

public string GetName()
{
    return name;
}

public void SetName(string name)
{
    this.name = name;
}

public string GetLastname()
{
    return lastname;
}

public void SetLastname(string lastname)
{
    this.lastname = lastname;
}

public void SayHi()
{
    Console.WriteLine($"Hello, my name is {GetFullName()}");
}

private string GetFullName()
{
    return $"{GetName()} {GetLastname()}";
}
}
```

Ejemplo 4.1

Creación de un objeto

Para crear un objeto se emplea la palabra reservada **new** seguido del nombre de la clase a instanciar.

Como es habitual en los lenguajes de programación, debe declararse una variable que se corresponda con el tipo de dato de la clase a crear, ya que será allí donde se almacene la referencia del objeto que la instrucción **new** provea.

Dicho con otras palabras, la instrucción reservará el espacio para el tipo de objeto en cuestión y devolverá la dirección de memoria, la referencia, que deberá ser almacenada en una variable para poder manipularla posteriormente.

```
Person myVariable = new Person();
```

Ejemplo 5.1

En el ejemplo expuesto, **myVariable** *referencia* a un objeto del **tipo Person**, de modo tal que mediante dicha variable ahora **será posible acceder a cualquiera de los miembros declarados como públicos**.

El acceso a los miembros se realiza a través del uso de un **punto** (de allí que reciben el nombre de accessor); en el ejemplo siguiente se muestra la secuencia completa para asignar el nombre de la persona, el apellido e invocar a la función que imprimirá en pantalla el saludo.

```
Person myVariable = new Person();  
myVariable.SayHi();
```

Ejemplo 5.2

Constructor

El uso de la instrucción **new** tiene por objetivo reservar el espacio necesario en la memoria RAM para alojar una instancia de la clase que se quiere crear; ejecutando la función que se conoce como **constructor**.

El constructor está sujeto a las mismas reglas que el resto de las funciones salvo que posee una diferencia muy particular: su firma se caracteriza por la ausencia del tipo de dato que devuelve.

Dado que siempre se ejecuta, es el espacio ideal para incluir instrucciones que sean de carácter obligatorio para la creación de un objeto, como puede ser la inicialización de alguna variable o el cumplimiento de ciertas condiciones.

Si no existe necesidad de realizar operaciones iniciales, puede crearse una clase omitiendo el constructor ya que implícitamente todo objeto requiere de uno y se asume su existencia; en cualquier otro escenario su presencia es requerida y puede definirse con parámetros o sin ellos.

La creación de un objeto haciendo uso del constructor predeterminado se evidencia por los paréntesis vacíos. Pero podemos tener presente que este constructor debe ser definido explícitamente en caso de que haya otro constructor ya definido. Es decir, si

definimos al menos un constructor, ya no estará disponible el constructor vacío como default y deberemos agregarlo.

```
Person myVariable = new Person();
```

Ejemplo 6.1

Si fuera obligatorio contar con el nombre y el apellido, por ejemplo, se podría realizar el siguiente ajuste en la clase Person.

```
using System;

namespace MyApp
{
    public class Person
    {
        private string name;
        private string lastname;

        public Person()
        {
            this.name = "Juan";
            this.lastname = "Perez";
        }

        public Person(string name, string lastname)
        {
            this.name = name;
            this.lastname = lastname;
        }

        public string GetName()
        {
            return name;
        }

        public void SetName(string name)
        {
            this.name = name;
        }

        public string GetLastname()
        {
            return lastname;
        }

        public void SetLastname(string lastname)
        {
            this.lastname = lastname;
        }
    }
}
```



```
public void SayHi()
{
    Console.WriteLine($"Hello, my name is {GetFullName()}");
}

private string GetFullName()
{
    return $"{GetName()} {GetLastname()}";
}
}
```

Ejemplo 6.2

La construcción del objeto nos obligaría a pasar el nombre y el apellido por parámetro.

```
using System;
namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Person myVariable = new Person("Al", "Goritmo");
            myVariable.SayHi();
        }
    }
}
```

Ejemplo 7.1

Es importante recordar que, si la intención es crear un objeto con determinados valores e impedir que se cambien a posteriori, la clase no debería contar con los setters de los atributos afectados.

El uso de la palabra *this*

En C#, Typescript, Java, C++ y en muchos otros lenguajes de programación, que implementan el paradigma orientado a objetos, se emplea la palabra reservada ***this*** para hacer **autorreferencia** a la instancia (o de la clase si es que se está hablando desde el punto de vista del diseño).



En el ejemplo 6.2 se observa cómo el constructor recibe los parámetros **name** y **lastName** y almacena los valores en sendas variables homónimas; ante el uso de elementos con el mismo nombre, la palabra reservada **this** permite identificar cuál es aquel que pertenece a la clase ya que, de otro modo, el compilador no sabría si la asignación se quiere realizar de los miembros hacia las variables de la función o viceversa.

Espacios de nombre

Al momento de abordar un proyecto en C# es necesario hacer hincapié en la **modularización** del mismo, ya que esto facilita la legibilidad y mantenibilidad. Junto con la reutilización del código, forman la base para un desarrollo más ordenado y veloz.

Uno de los factores que contribuye a esto es la organización de las clases en **namespaces**, que no son ni más ni menos que contenedores lógicos que agrupan clases bajo un criterio general. En la práctica, los namespaces suelen coincidir con la estructura de carpetas del proyecto, pero lo importante es que permiten organizar y evitar conflictos de nombres.

Por ejemplo, podrías tener un namespace destinado a clases de **modelos de datos**, otro para **servicios de validación**, y otro para **conexiones con APIs externas**. La nomenclatura utilizada para los namespaces, así como la organización del código fuente, debe estar guiada por la necesidad y el criterio del arquitecto o equipo de desarrollo.

Importación de clases

Cuando desde una clase se hace referencia a otra que no se encuentra en el mismo namespace, ya sea para instanciarla o para hacer uso de sus miembros públicos estáticos, es preciso indicarle al compilador dónde se encuentra.

En C# esto se logra mediante la palabra reservada **using**, seguida del nombre del namespace que contiene la clase a referenciar.

```

using System;

namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Person myVariable = new Person();
            myVariable.SetName("Mariano");
            myVariable.SetLastname("Kaimakamian Carrau");
            myVariable.SayHi();
        }
    }
}

```

Ejemplo 9.1

Sobrecarga

Del inglés **overloading**, se trata de una palabra que hace referencia a la capacidad que posee el paradigma orientado a objetos de permitir, dentro de una misma clase, la **existencia de funciones con la misma firma**; la única condición es que los **parámetros deben diferir en su tipo de datos, su cantidad y/o ordinalidad de tipos**.

Esta característica permite ordenar el código al emplear un único nombre para funciones que realizan tareas similares pero con parámetros de entrada distintos.

Visibilidad	Tipo	Nombre	Parámetros
public	string	concatPersonInfo	(string name, string lastName)
public	string	concatPersonInfo	(string name, string middleName, string lastName)
public	string	concatPersonInfo	(string name, string lastName, int age)

Ejemplo 10.1

En este ejemplo existen tres métodos con el mismo nombre (**ConcatPersonInfo**) pero con diferentes parámetros. El compilador selecciona automáticamente la versión correcta según los argumentos utilizados.

En el ejemplo siguiente se observa un caso que impide la compilación del código ya que se está frente a la existencia de dos funciones de firmas idénticas; el compilador no distingue los nombres de los parámetros sino que analiza la coincidencia basándose en los tipos.

Visibilidad	Tipo	Nombre	Parámetros
public	string	getClientInfo	(int account)
public	string	getClientInfo	(int id)

Ejemplo 10.2

Diferencia entre CSharp y un lenguaje no tipado

En lenguajes fuertemente tipados como **C#**, **Java** o **C++**, la sobrecarga funciona de manera natural: el compilador puede identificar cuál método invocar en base a los tipos y cantidad de parámetros.

En lenguajes dinámicos como **Python** o **PHP**, esto no resulta tan sencillo porque carecen de tipos estrictos. Allí el programador debe implementar lógica interna para distinguir los casos (por ejemplo, usando parámetros opcionales o estructuras como `...args`).

```
class Program
{
    static void Main(string[] args)
    {
        string fullName = PersonInfo.ConcatPersonInfo("John", "Doe");
        Console.WriteLine(fullName); // Output: John Doe

        string fullMiddleName = PersonInfo.ConcatPersonInfo("John", "Quincy",
"Adams");
        Console.WriteLine(fullMiddleName); // Output: John Quincy Adams

        string personWithAge = PersonInfo.ConcatPersonInfo("Jane", "Smith", 30);
        Console.WriteLine(personWithAge); // Output: Jane Smith, 30 years old
    }
}
```

Ejemplo 10.3

Como podemos observar, tenemos 3 métodos con el mismo nombre, al momento de invocar a cada uno de ellos, el compilador puede identificar en base a los parámetros utilizados en dicha invocación a cuál de las 3 versiones del método estamos invocando.

En lenguajes no tipados, o dinámicamente tipados como Typescript, Python, PHP, etc. esto no resulta tan sencillo ya que justamente carecemos de tipos de datos. Para lograr el mismo resultado utilizando Typescript, el mecanismo de identificación queda a cargo de quien desarrolle la función. Veamos una posible implementación (hay otras formas de lograrlo):

```
using System;

namespace MyApp
{
    public class Person
    {
        // Sobrecarga de métodos en C#
        public static string ConcatPersonInfo(string name, string lastName)
        {
            return $"{name} {lastName}";
        }

        public static string ConcatPersonInfo(string name, string middleName,
string lastName)
        {
            return $"{name} {middleName} {lastName}";
        }

        public static string ConcatPersonInfo(string name, string lastName, int
age)
        {
            return $"{name} {lastName}, {age} years old";
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            string fullName = Person.ConcatPersonInfo("John", "Doe");
            Console.WriteLine(fullName); // Output: John Doe

            string fullMiddleName = Person.ConcatPersonInfo("John", "Quincy",
"Adams");
            Console.WriteLine(fullMiddleName); // Output: John Quincy Adams

            string personWithAge = Person.ConcatPersonInfo("Jane", "Smith", 30);
            Console.WriteLine(personWithAge); // Output: Jane Smith, 30 years old
        }
    }
}
```

```

    }
}

```

Ejemplo 10.4

Herencia

La **herencia** es el nombre que recibe la característica de la programación orientada a objetos que **permite crear una clase utilizando a otra existente como base, estableciendo una relación de sucesión en la que la nueva entidad puede hacer uso pleno de los miembros públicos y protegidos de su antecesor, como si fueran suyos.**

Definición	Una clase derivada, posee los atributos y métodos públicos y protegidos de su clase base.
------------	---

C# sólo permite herencia simple, al igual que Java, aunque existen otros lenguajes que admiten heredar de múltiples clases como por ejemplo C++.

Para definir una herencia es requisito utilizar en la definición de la clase dos puntos “:”, **seguido del nombre de la clase de la que se quiere extender.**

En los siguientes ejemplos, **las clases Employee y ProviderEmployee extienden la clase Person** bajo una misma premisa: un empleado es una persona y, por transitividad, posee los mismos atributos que la definen (nombre, segundo nombre, apellido y edad).

```

public class Employee : Person
{
    public int File { get; set; }
    public int LaborOld { get; set; }
    public string Career { get; set; }
    // resto de los miembros...
}

```

Ejemplo 11.1

```

public class ProviderEmployee : Person
{
    public int PermissionCode { get; set; }
    public int CompanyCode { get; set; }
}

```

```
// resto de los miembros...  
}
```

Ejemplo 11.2

Cabe destacar que tanto la clase *Employee* como *ProviderEmployee* tendrán los atributos y métodos públicos y protegidos declarados en la clase *Person*.

Habitualmente los problemas suelen requerir modelar elementos que presentan características comunes y estrategias como el uso de la herencia contribuyen a la mantenibilidad y legibilidad del código al evitar que se duplique, además de ordenar conceptualmente las entidades.

Upcasting y Downcasting

La implementación de la herencia abre las puertas al uso del **upcasting** y el **downcasting**, permitiendo que una variable pueda ser tratada como alguno de sus antecesores de los que deriva o bien, cumpliendo ciertos requisitos, que una variable pueda ser tratada como alguno de sus sucesores.

```
public class Person  
{  
    public void SayHi() => Console.WriteLine("hi");  
}  
  
public class Employee : Person  
{  
    public void SayBye() => Console.WriteLine("bye");  
}  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        Person employee = new Employee(); // upcasting implícito  
    }  
}
```

Ejemplo 12.1

En el ejemplo anterior se exhibe un **upcasting** implícito; nótese cómo un objeto de tipo *Employee* (clase derivada) es asignado a una variable de tipo *Person* (clase base).

Otro modo de realizar el upcasting es mediante el casteo explícito, como se muestra a continuación, en el que se crean instancias de *Employee* y *ProviderEmployee*, para luego ser reasignadas a variables del tipo *Person*.

```
class Program
{
    static void Main(string[] args)
    {
        Employee employee = new Employee();
        Person person = (Person)employee; // upcasting explícito
    }
}
```

Ejemplo 13.1

El uso del **upcasting** es importante ya que **habilita el tratamiento de los objetos de modo más genérico**; por ejemplo, si una empresa quisiera agasajar a todas las personas que contribuyen con su crecimiento y admitiera a los proveedores, todos serían tratados como *Person*.

Cabe aclarar que **al momento de hacer un upcasting**, que no necesariamente tiene que remitir a la clase inmediatamente superior, **se pierde acceso a todos los miembros característicos de la subclase**.

El **downcasting** no permite el casteo a otro tipo de dato más que **al mismo de la instancia original**.

```
class Program
{
    static void Main(string[] args)
    {
        Person employeePerson = new Employee(); // upcasting
        Employee employee = (Employee)employeePerson; // downcasting
    }
}
```

Ejemplo 13.2

El ejemplo 13.2 muestra un upcasting implícito y luego un downcasting explícito; esta última acción es posible porque originalmente se trataban de instancias más específicas,

lo que implica que al momento de crear los objetos se reservó el espacio necesario para todos sus miembros.

El upcasting no produce una nueva reasignación de memoria, sino que se limita a referenciar al objeto abarcando únicamente los miembros propios de su tipo.

Sobreescritura

Otra característica muy ligada a la herencia de lo que en inglés se conoce como **overriding**, una práctica mediante la cual un miembro público o protegido declarado en una clase padre, es invalidado para que tome precedencia la de aquella subclase que se quiere instanciar.

De este modo, ***es posible redefinir el comportamiento de las clases que se extienden de otras***, manteniendo el qué hacen pero no el cómo lo hacen, ***lo que resulta en reaccionar de un modo distinto ante un mismo llamado.***

Para conseguir la sobreescritura, simplemente debe reimplementarse el método en la subclase deseada, respetando la firma..

Un ejemplo trivial, pero no por eso menos válido, podría ser el considerar que los empleados de una compañía saludan indicando su nombre completo y número de legajo, mientras que los proveedores requieren indicar su apellido y el id de la compañía a la que pertenecen.

Notar que ***se ha cambiado el nivel de visibilidad del método getFullName() por protected***, ya que la clase Employee requiere el uso del nombre completo; al existir el código que produce ese efecto, no es necesario duplicar código: este ligero cambio es un ejemplo del potencial de la programación orientada a objetos en términos de reutilización.

```
using System;

namespace MyApp
{
    public class Person
    {
        // ... otras propiedades/métodos

        public virtual void SayHi()
        {
            Console.WriteLine($"Hello, my name is {GetFullName()}");
        }
    }
}
```

```

    }

    protected virtual string GetFullName()
    {
        return $"{GetName()} {GetLastname()}";
    }

    // Supongamos que existen estos métodos auxiliares
    public string GetName() => "Nombre";
    public string GetLastname() => "Apellido";
}

```

Ejemplo 14.1

Los fragmentos de código 15.1 y 15.2 muestran la sobreescritura, realizando los cambios pertinentes.

```

public class Employee : Person
{
    public int File { get; set; }

    public override void SayHi()
    {
        Console.WriteLine($"Hello, my name is {GetFullName()} and my file is {File}");
    }
}

```

Ejemplo 15.1

```

public class ProviderEmployee : Person
{
    public int CompanyCode { get; set; }

    public override void SayHi()
    {
        Console.WriteLine($"Hello, my name is {GetLastname()} and my company's id is {CompanyCode}");
    }
}

```

Ejemplo 15.2

El resultado final se resume en la existencia de tres clases, relacionadas por herencia, que ante el llamado de la misma función producen resultados diferentes.

Es importante tener en cuenta que la implementación de la **sobreescritura** no es obligatoria para todas las subclases; simplemente es un instrumento que permite adaptar el código a las necesidades de la solución mediante cambios acotados.

Otro detalle fundamental es que, aun cuando se haya producido un **upcasting**, una función sobreescrita se comportará como se haya definido en la subclase debido al mecanismo de sobreescritura.

El uso de la palabra base

Los motivos por los que se puede considerar la sobreescritura de un método son:

- Reemplazar el código
- Extender funcionalidad
- Constructores

El primero de los casos es el más trivial y se da cuando el comportamiento de la subclase debe cambiar por completo. El segundo se presenta cuando la subclase requiere exactamente el mismo algoritmo que el definido en la clase base, pero con un agregado funcional.

Dado que duplicar código no favorece la mantenibilidad, la programación orientada a objetos en C# ofrece una alternativa: utilizar la palabra reservada **base** para hacer referencia a la clase padre y acceder a sus miembros.

Retomando el ejemplo anterior, si consideramos la función **SayHi()** de la clase **Person** y la sobreescritura en **Employee**, la única diferencia entre ellas es que la última agrega un mensaje personalizado. Conociendo la existencia de la palabra reservada **base**, ahora es posible reescribir el código de manera más prolija y reutilizable.

```
using System;

namespace MyApp
{
    public class Person
    {
        public static string ConcatPersonInfo(string name, string lastName)
```

```

    {
        return $"{name} {lastName}";
    }

    public static string ConcatPersonInfo(string name, string middleName,
string lastName)
    {
        return $"{name} {middleName} {lastName}";
    }

    public static string ConcatPersonInfo(string name, string lastName, int
age)
    {
        return $"{name} {lastName}, {age} years old";
    }

    public static void Main(string[] args)
    {
        string fullName = ConcatPersonInfo("John", "Doe");
        Console.WriteLine(fullName); // Output: John Doe

        string fullMiddleName = ConcatPersonInfo("John", "Quincy", "Adams");
        Console.WriteLine(fullMiddleName); // Output: John Quincy Adams

        string personWithAge = ConcatPersonInfo("Jane", "Smith", 30);
        Console.WriteLine(personWithAge); // Output: Jane Smith, 30 years old
    }
}

```

Ejemplo 16.1

El efecto de este cambio resultará en la ejecución de la función `SayHi()` de la clase padre, para luego continuar con la línea de la clase `Employee`.

Es importante advertir que la llamada a un miembro mediante la palabra reservada `base` no requiere necesariamente que se realice al inicio de la función; no existe restricción alguna más que la necesidad de lo que se desea realizar.

El tercer caso resulta mandatorio cuando la clase padre presenta al menos un **constructor personalizado**, ya que la subclase siempre debe ser capaz de llamar a alguno de los constructores de su padre.

Considerar el siguiente caso:

```
using System;

namespace MyApp
{
    public class Person
    {
        public string Name { get; set; }
        public string Lastname { get; set; }

        public Person(string name, string lastname)
        {
            Name = name;
            Lastname = lastname;
        }

        public virtual void SayHi()
        {
            Console.WriteLine($"Hello, my name is {Name} {Lastname}");
        }
    }

    public class Employee : Person
    {
        public int File { get; set; }

        public Employee(string name, string lastname, int file)
            : base(name, lastname)
        {
            File = file;
        }

        public override void SayHi()
        {
            Console.WriteLine($"Hello, my name is {Name} {Lastname} and my file is {File}");
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            Employee emp = new Employee("Al", "Goritmó", 123);
            emp.SayHi();
        }
    }
}
```

Ejemplo 17.1

La clase **Person** presenta dos constructores.

La clase **Employee** está obligada a implementar al menos una llamada a alguno de los constructores que posee su clase base; en su defecto, debe definir un constructor que ejecute **base(name)** o **base(name, age)**.

```
public class Employee : Person
{
    public Employee(string name)
        : base(name) // llamada al constructor de la clase padre
    {
    }

    public Employee(string name, int age)
        : base(name, age) // llamada al otro constructor de la clase padre
    {
    }
}
```

Ejemplo 17.2

Clase abstracta

Las clases que se denominan **abstractas** difieren de las concretas por presentar dos particularidades, siendo una de ellas la **imposibilidad de ser instanciada** y, la otra, que **puede contener funciones con implementación o sin ellas** (abstractas).

Su existencia obedece a la necesidad de poder contar con una entidad que **reúna un conjunto de características necesarias para las subclases, pero no suficientes como para que esta clase en cuestión posea identidad propia (ser instanciada)**, limitando dicha responsabilidad a las subclases que de ella se extiendan.

Por ejemplo, si se requiriese desarrollar una solución para una cadena de estacionamientos, podría ser requisito modelar autos, camiones, bicicletas y motos, concentrando la mayor cantidad de características comunes en una clase base llamada Vehículo.

Por su naturaleza no tendría sentido poder crear un objeto Vehículo ya que es una representación muy genérica en comparación a las subclases, de modo tal que sería

candidata para definirla como abstracta y evitar, de este modo, que se instancie inadvertidamente.

Definir si una clase amerita ser abstracta o no, depende de las necesidades del proyecto y la experiencia de quien diseñe la solución.

```
public abstract class Shape
{
    public abstract double Area();
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public Rectangle(double width, double height)
    {
        Width = width;
        Height = height;
    }

    public override double Area()
    {
        return Width * Height;
    }
}
```

Ejemplo 19.1

El ejemplo anterior muestra la definición de una clase abstracta, con la declaración de las firmas de dos funciones abstractas y una tercera concreta.

Las funciones abstractas deben ser implementadas por las subclases, para lo cual deben sobrecribirse.

```
using System;

public abstract class Vehicle
{
    public abstract bool TurnEngineOn();
    public abstract bool TurnEngineOff();
}

public class Car : Vehicle
{
    public override bool TurnEngineOn()
    {
        Console.WriteLine("Encendiendo el motor del coche...");
    }
}
```

```
// Lógica para encender el motor
return true; // o false si falla
}

public override bool TurnEngineOff()
{
    Console.WriteLine("Apagando el motor del coche...");
    // Lógica para apagar el motor
    return true; // o false si falla
}
}
```

Ejemplo 20.1

Composición y agregación

Genéricamente hablando, la composición es la relación de dependencia que se establece entre dos objetos, en el que uno es el compuesto y el otro es el componente.

Con un poco más de formalidad, la composición se presenta cuando ***una clase posee al menos un atributo cuyo tipo de dato se corresponde con el de otra clase y no con el de un tipo primitivo.***

Existen dos tipos de relaciones:

- ***Débil (agregación):*** el objeto principal no requiere necesariamente de la existencia del componente.
- ***Fuerte (composición):*** el objeto principal no puede existir sin el componente.

Por ejemplo, construir un auto requiere entre otras cosas de la existencia de un motor y de un aire acondicionado; mientras la relación entre el auto y el motor es fuerte (composición) porque el auto no puede funcionar sin él, la relación entre el auto y aire acondicionado es débil (agregación) porque no es esencial para prestar el servicio.

A continuación, se exhiben las clases que podrían adecuarse al ejemplo.

Notar que ***AirConditioner*** y ***Engine*** presentan miembros ***TurnOn()*** y ***TurnOff()*** pero sin ningún motivo en particular; en este caso coincide el hecho de necesitar una función para el encendido y apagado.

```
using System;
```



```
namespace MyApp
{
    public class AirConditioner
    {
        // ... otros miembros

        public bool TurnOn()
        {
            Console.WriteLine("Encendiendo el aire acondicionado...");
            // Lógica para encender el aire acondicionado
            return true; // o false si falla el encendido
        }

        public bool TurnOff()
        {
            Console.WriteLine("Apagando el aire acondicionado...");
            // Lógica para apagar el aire acondicionado
            return true; // o false si falla el apagado
        }
    }

    class Program
    {
        static void Main(string[] args)
        {
            AirConditioner ac = new AirConditioner();
            ac.TurnOn();
            ac.TurnOff();
        }
    }
}
```

Ejemplo 21.1

```
using System;

namespace MyApp
{
    public class Engine
    {
        // ... otros miembros

        public bool TurnOn()
        {
            Console.WriteLine("Encendiendo el motor...");
            // Lógica para encender el motor
            return true; // o false si falla el encendido
        }
    }
}
```

```

    public bool TurnOff()
    {
        Console.WriteLine("Apagando el motor...");
        // Lógica para apagar el motor
        return true; // o false si falla el apagado
    }

    public void SpeedUp(int increase)
    {
        Console.WriteLine($"Aumentando la velocidad en {increase} unidades.");
        // Lógica para aumentar la velocidad del motor
    }
}

class Program
{
    static void Main(string[] args)
    {
        Engine engine = new Engine();
        engine.TurnOn();
        engine.SpeedUp(20);
        engine.TurnOff();
    }
}

```

Ejemplo 21.2

Habiendo definido los componentes, en el que el primero es débil por ser un “agregado” y el segundo fuerte por el carácter esencial, resta el compuesto.

```

using System;

namespace MyApp
{
    public abstract class Vehicle
    {
        private Engine engine;
        private AirConditioner airConditioner;

        protected Vehicle(Engine engine, AirConditioner airConditioner)
        {
            this.engine = engine;
            this.airConditioner = airConditioner;
        }

        public bool TurnEngineOn()
        {
            return engine.TurnOn();
        }
    }
}

```

```
    }

    public bool TurnEngineOff()
    {
        return engine.TurnOff();
    }

    public void SpeedUp(int increase)
    {
        engine.SpeedUp(increase);
    }

    public bool TurnAcOn()
    {
        return airConditioner.TurnOn();
    }

    public bool TurnAcOff()
    {
        return airConditioner.TurnOff();
    }

    // ... otros miembros
}
}
```

Ejemplo 22.1

Salvo casos muy particulares, toda composición y agregación implica el uso de algunos de los miembros de sus componentes, de allí que en el ejemplo se observa cómo Vehicle invoca a las funciones de sus componentes: encendido, apagado y aceleración.

Otro detalle importante a mencionar es que suele abusarse del término composición para referirse a la construcción de objetos en base a otros y no tanto al grado de dependencia.

La **composición** y la **agregación** son esenciales porque no sólo permiten la reutilización del código, sino que **aportan valor agregado al polimorfismo ya que un objeto compuesto, en tiempo de ejecución, puede variar su comportamiento mediante la reasignación de sus componentes**.

Interfaz

La identidad, los atributos y las funciones son los tres elementos que definen a un objeto. En particular, las funciones son responsables de modificar el estado de los mismos; de allí que existen diversas estrategias centradas en torno al comportamiento: sobrecarga, sobreescritura, composición y herencia.

Un cuarto recurso es lo que se conoce como **interfaz**. Una interfaz define las **firmas de los métodos** que debe implementar una clase cuando decide ajustarse a ella.

Mientras una clase (concreta o abstracta) define atributos y comportamientos, una interfaz especifica únicamente el **contrato de comportamiento** que deben cumplir los objetos, incluso si no forman parte de la misma jerarquía.

Retomando el ejemplo de la composición, tanto el aire acondicionado como el motor presentan las funciones `TurnOn()` y `TurnOff()`. Aunque son objetos de distintas clases, ambos pueden encenderse y apagarse, lo que permite tratarlos como elementos “encendibles” y “apagables”.

Las interfaces únicamente definen las firmas de los métodos a ser implementados por las clases.

```
public interface ITurnable
{
    bool TurnOn();
    bool TurnOff();
}
```

Ejemplo 23.1

Las clases que implementan una interfaz en **C#** deben declarar la interfaz en la definición de la clase usando el operador `:` seguido del nombre de la interfaz. En caso de implementar más de una, los nombres deben separarse con comas.

Al igual que ocurre con la sobreescritura convencional, los métodos definidos en la interfaz **deben ser implementados obligatoriamente** por la clase.

```
using System;

public class AirConditioner : ITurnable
{
    public bool TurnOn()
    {
        Console.WriteLine("Encendiendo el aire acondicionado...");
        return true; // o false si falla
    }

    public bool TurnOff()
    {
        Console.WriteLine("Apagando el aire acondicionado...");
        return true; // o false si falla
    }
}
```

Ejemplo 24.1

```
using System;

public class Engine : ITurnable
{
    public bool TurnOn()
    {
        Console.WriteLine("Encendiendo el motor...");
        return true; // o false si falla
    }

    public bool TurnOff()
    {
        Console.WriteLine("Apagando el motor...");
        return true; // o false si falla
    }
}
```

Ejemplo 24.2

Al igual que ocurre con las clases, en el que **un objeto puede manipularse considerando** su tipo de dato o como si fuera alguno de sus ancestros (upcasting), también es posible tratarlos según **cualquiera de las interfaces que implemente**.

Suele ser deseable programar pensando en interfaces ya que esto permite no ajustarse a un objeto en particular, sino a entidades que se comportan de un determinado modo, **permitiendo que -en caso de ser necesario- pueda sustituirse el objeto subyacente por otro que se comporte de la misma manera**, evitando realizar modificaciones importantes en el código.

Polimorfismo

En un sentido práctico aplicado a la programación, **se trata de la capacidad que poseen los objetos de reaccionar de diversas maneras ante un mismo llamado, lo que ocurre como resultado de implementar comportamientos distintos en funciones que respetan la misma firma**.

Esto surge de la mano de la **sobreescritura**, la implementación de **interfaces** y la **composición** de objetos.

La sobreescritura permite el polimorfismo dentro de una misma jerarquía, mientras que la implementación de una **interfaz (o en su defecto extender una clase) permite involucrar objetos de distinta naturaleza**.

La **composición**, en cambio, **habilita que un objeto adopte distinto comportamiento en tiempo de ejecución**, según la implementación del objeto que lo compone, sin considerar la naturaleza de la jerarquía a la que pertenece el compuesto.

Colecciones

Las colecciones son objetos que tienen la capacidad de **agrupar elementos del mismo tipo** y ofrecen operaciones para incorporarlos, eliminarlos y recorrerlos.

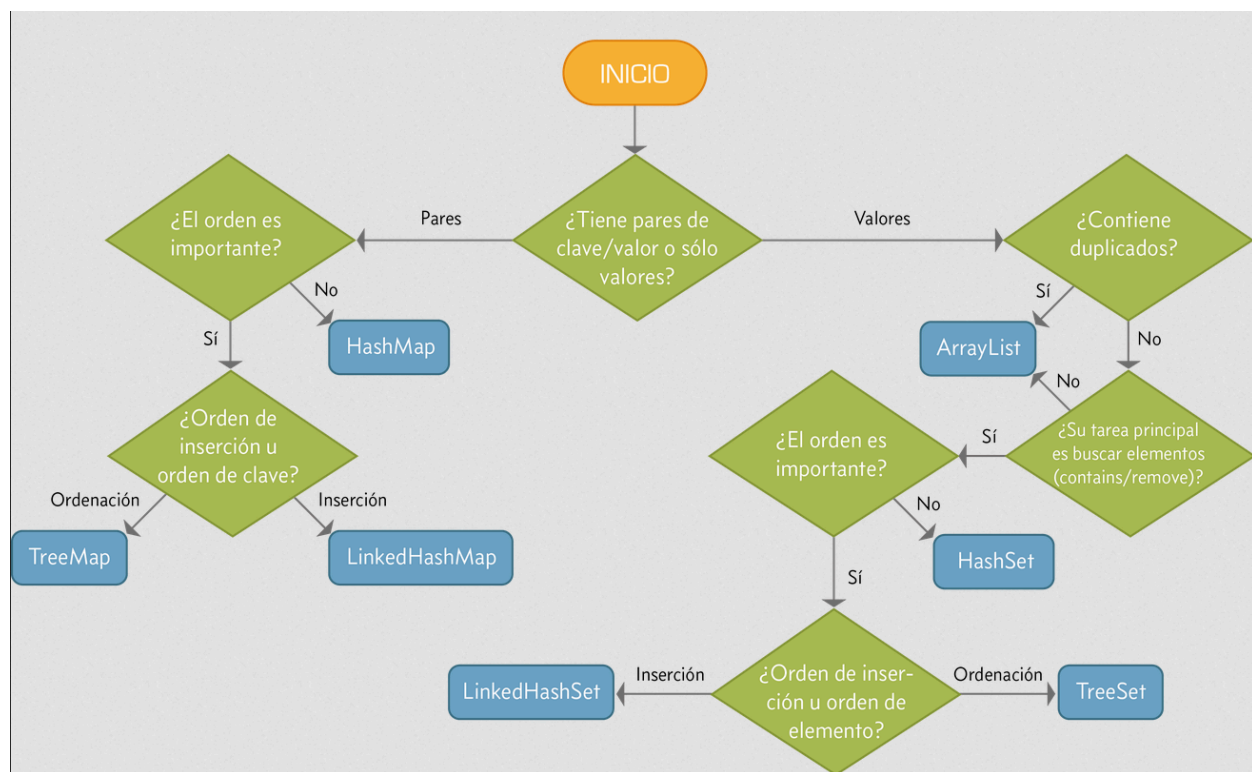
A diferencia de un **array** (arreglo), que tiene tamaño fijo y un manejo más básico, las colecciones en C# como **listas, diccionarios y conjuntos** poseen la facultad de **autodimensionarse** y ofrecen una experiencia más flexible, aunque con un costo adicional en recursos.

C#

En C# las colecciones de las que disponemos son:

- **List** → colección secuencial, flexible y muy usada.
- **Dictionary** → colección de pares clave-valor, similar a **Map**.
- **HashSet** → colección que garantiza unicidad de elementos, similar a **Set**.
- Todas estas colecciones están en el **namespace** **System.Collections.Generic**.

Para utilizar cada una de estas colecciones, debemos indicar el tipo de dato que almacenará.



Fuente: <https://www.adictosaltrabajo.com/2015/09/25/introduccion-a-colecciones-en-java/>

List

Es el tipo de colección más simple y el más utilizado (probablemente). Este tipo de colección nos permite definir una lista de elementos de tamaño variable.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        // Lista de números
        List<int> numeros = new List<int>();

        // Agregar elementos
        numeros.Add(1);
        numeros.Add(10);

        // Remover el último elemento
        numeros.RemoveAt(numeros.Count - 1);

        // Recorrer la lista
        foreach (int numero in numeros)
        {
            Console.WriteLine(numero);
        }
    }
}
```

Ejemplo 25.1

Para más información sobre las operaciones que se pueden realizar sobre un List consultar: [List](#)

Dictionary

Esta estructura de datos permite almacenar elementos de tipo clave - valor. La característica distintiva es que una clave sólo puede aparecer una vez. Podemos pensar esta estructura como un diccionario.

Cabe destacar que tanto las claves como los valores pueden ser de cualquier tipo, inclusive objetos personalizados.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
```



```
// Diccionario con clave string y valor int
Dictionary<string, int> agenda = new Dictionary<string, int>();

// Agregar elementos
agenda["Juan"] = 1234;
agenda["Ana"] = 5678;

// Obtener un elemento
Console.WriteLine(agenda["Ana"]);

// Saber si existe una clave
Console.WriteLine(agenda.ContainsKey("Ana"));

// Eliminar un elemento
agenda.Remove("Juan");

// Recorrer el diccionario
foreach (var kvp in agenda)
{
    Console.WriteLine($"Nombre: {kvp.Key}, Número: {kvp.Value}");
}
}
```

Ejemplo 25.2

Para más información sobre las operaciones que se pueden realizar sobre un Dictionary consultar: [Dictionary](#)

HashSet

Como mencionamos anteriormente, un Set es un tipo de estructura de datos que se caracteriza por no admitir duplicados.

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        // Conjunto de números únicos
        HashSet<int> unicos = new HashSet<int>();

        // Agregar elementos
        unicos.Add(1234);
        unicos.Add(5678);
        unicos.Add(1234); // Ignorado, ya existe
    }
}
```

```

        // Saber si existe un elemento
        Console.WriteLine(unicos.Contains(1234));

        // Eliminar un elemento
        unicos.Remove(1234);

        // Recorrer el conjunto
        foreach (int unico in unicos)
        {
            Console.WriteLine(unico);
        }
    }
}

```

Ejemplo 25.3

En **C#**, para que un **HashSet** funcione con objetos propios se debe redefinir el método **Equals()** y **GetHashCode()**, de ese modo el **HashSet** podrá determinar la unicidad correctamente.

```

using System;
using System.Collections.Generic;
using System.Linq;

public class GenericSet<T>
{
    private List<T> items;
    private Func<T, string> getKey;

    public GenericSet(Func<T, string> comp)
    {
        getKey = comp;
        items = new List<T>();
    }

    public GenericSet(Func<T, string> comp, IEnumerable<T> values)
    {
        getKey = comp;
        items = new List<T>();
        foreach (var v in values)
        {
            Add(v);
        }
    }

    public bool Has(T item)
    {
        return items.Any(x => getKey(x) == getKey(item));
    }
}

```

```
public T Get(string key)
{
    return items.FirstOrDefault(x => getKey(x) == key);
}

public void Add(T item)
{
    if (!Has(item))
    {
        items.Add(item);
    }
}

public void Delete(T item)
{
    if (Has(item))
    {
        items = items.Where(x => getKey(x) != getKey(item)).ToList();
    }
}

public IEnumerable<T> Values()
{
    return items.ToList();
}

public void ForEach(Action<T, int, List<T>> cb)
{
    for (int i = 0; i < items.Count; i++)
    {
        cb(items[i], i, items);
    }
}
}
```

Ejemplo 25.4

Veamos un ejemplo de su uso

```
public class Persona
{
    public int Id { get; }
    public string Name { get; }

    public Persona(int id, string name)
    {
        Id = id;
        Name = name;
    }
}
```

```
public string SayHi() => $"Hola {Name}";
public string GetKey() => Id.ToString();

public override string ToString() => $"{Id} - {Name}";
}

class Program
{
    static void Main()
    {
        var unicos = new GenericSet<Persona>(p => p.GetKey());

        unicos.Add(new Persona(1, "Juan"));
        unicos.Add(new Persona(1, "Juan")); // Ignorado, ya existe

        foreach (var p in unicos.Values())
        {
            Console.WriteLine(p);
        }

        Console.WriteLine("-----");

        var p3 = new Persona(3, "Ana");
        var arrayUnicos = new List<Persona>
        {
            new Persona(1, "Pedro"),
            new Persona(2, "Fede"),
            p3,
            new Persona(4, "Laura")
        };

        var unicos2 = new GenericSet<Persona>(a => a.GetKey(), arrayUnicos);

        unicos2.ForEach((x, i, arr) => Console.WriteLine($"{i}: {x}"));

        unicos2.Delete(p3);
        unicos2.ForEach((x, i, arr) => Console.WriteLine($"{i}: {x}"));

        Console.WriteLine("-----");
        Console.WriteLine($"Un elemento: {unicos2.Get("2")}");
    }
}
```

Ejemplo 25.5

El uso de la palabra Static

Para poder entender las implicancias de esta palabra reservada, es importante tener en cuenta que ***todo miembro que requiera de la existencia de un objeto para ser accedido, se lo conoce como miembro de instancia, mientras que aquel que no lo necesite, se lo conoce como miembro de clase.***

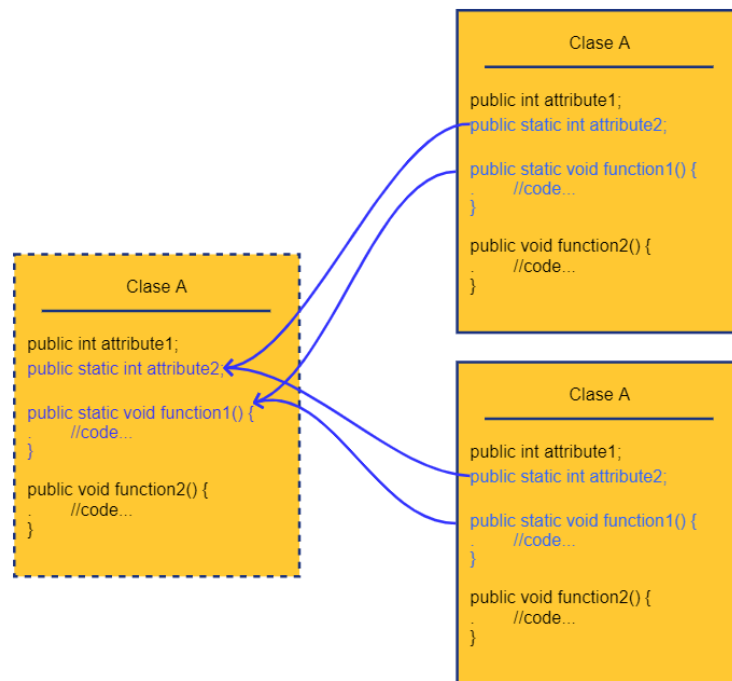
La palabra ***static*** es la directiva que ***se emplea para designar a los miembros de clase,*** lo que implica que ***pueden ser invocados sin necesidad de que exista previamente una instancia.***

Retomando conceptos fundamentales, considerar que cada vez que se ejecuta un programa, se reserva suficiente espacio en memoria como para hacer una copia de todas las clases que se van a utilizar durante el ciclo de vida del programa, creando un repositorio virtual.

Luego, por cada nueva instancia que se necesite crear, el sistema operativo buscará en dicho repositorio la clase de la que se requiere una copia y procederá a su clonación, es decir, se reservará un nuevo espacio en memoria suficiente como para alojar la instancia.

Mientras todos los miembros no estáticos serán clonados al nuevo espacio, de aquellos marcados como estáticos únicamente se copiarán las referencias al espacio original, es decir, a la clase del repositorio virtual.

El siguiente gráfico ejemplifica conceptualmente lo que ocurre cuando se realizan instancias de una clase que presenta miembros estáticos; tanto `attribute2` como `function1` estarán apuntando al espacio de memoria original, mientras que el resto de los miembros serán alocados en su propio espacio.



Implicancia de los miembros estáticos

Como ocurre con todas las variables que apuntan a un mismo sector de memoria, cualquier cambio que suceda en éste será visible por todos los miembros que lo referencien debido a la centralización.

En el caso de las funciones este efecto no se advierte porque las instancias de un mismo tipo de clase siempre ejecutarán el mismo comportamiento, y cualquier subclase que se extienda y sobrescriba una función estática estará teniendo precedencia sobre el método original.

Es de vital importancia entender que **todo miembro no estático puede acceder a un miembro estático pero no al revés**, ya que se presenta una suerte de relación uno a muchos; luego, los miembros estáticos pueden interactuar entre sí.

El siguiente ejemplo muestra cómo declarar una función estática.

```
using System;

public class MyClass
{
```

```
public static bool PerformHandshake()
{
    Console.WriteLine("Realizando handshake...");
    // Lógica para realizar el handshake
    return true; // o false si falla
}

class Program
{
    static void Main()
    {
        // Invocación sin necesidad de crear una instancia
        bool result = MyClass.PerformHandshake();
        Console.WriteLine($"Handshake result: {result}");
    }
}
```

Ejemplo 31.1

A continuación se aprecia la declaración de dos atributos estáticos; estos aún pueden ser alterados ya que la palabra reservada `static` solamente indica si pertenece al objeto o a la clase.

```
class Modem {
    public static int BAUD_RATE_9600 = 9600;
    public static int BAUD_RATE_115200 = 115200;

    // ... otros miembros
}
```

Ejemplo 31.2

Declaración de constantes

Comúnmente los atributos estáticos se emplean para definir constantes; combinar las palabras `readonly` y `static` en la declaración de una variable, resultará en la creación de un atributo que no puede ser alterado y que existe en un sólo lugar en memoria.

Un ligero cambio al ejemplo 31.2 permitirá que existan los atributos a nivel clase y que sea imposible modificarla, características que suelen ser convenientes para una constante.

```
public class Modem
{
    public static readonly int BAUD_RATE_9600 = 9600;
    public static readonly int BAUD_RATE_115200 = 115200;

    // ... otros miembros
}

class Program
{
    static void Main()
    {
        Console.WriteLine($"Baud rate estándar: {Modem.BAUD_RATE_9600}");
        Console.WriteLine($"Baud rate rápido: {Modem.BAUD_RATE_115200}");
    }
}
```

Ejemplo 31.3

Enumerados

Son objetos que agrupan literales con el propósito de ser empleados como constantes y resultan ser una opción más formal que los atributos estáticos y finales; como es de esperarse, ***las clases del tipo enum no se pueden extender ni modificar dado que se emplean para tipar aspectos dentro de un rango de valores fijos.***

Por convención, toda constante debe ser escrita en mayúscula, separando las palabras que componen su literal mediante el uso de guiones bajo; en el ejemplo 31.3, en el que se declaran constantes utilizando atributos estáticos y finales, se aprecia claramente.

```
enum DayOfWeek {
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday,
}
```


Ejemplo 32.1

Su uso es muy sencillo.

```
using System;

class Program
{
    static void Main()
    {
        DayOfWeek selectedDay = DayOfWeek.Monday;

        switch (selectedDay)
        {
            case DayOfWeek.Monday:
                Console.WriteLine("Es lunes");
                break;
            case DayOfWeek.Tuesday:
                Console.WriteLine("Es martes");
                break;
            case DayOfWeek.Wednesday:
                Console.WriteLine("Es miércoles");
                break;
            case DayOfWeek.Thursday:
                Console.WriteLine("Es jueves");
                break;
            case DayOfWeek.Friday:
                Console.WriteLine("Es viernes");
                break;
            case DayOfWeek.Saturday:
                Console.WriteLine("Es sábado");
                break;
            case DayOfWeek.Sunday:
                Console.WriteLine("Es domingo");
                break;
            default:
                Console.WriteLine("Día desconocido");
                break;
        }
    }
}
```

Ejemplo 32.2

Errores y excepciones

En el ámbito del desarrollo, existen tres tipos de causas por las que una pieza de software puede fallar:

- **Errores sintácticos y semánticos**

- **Errores de lógica**
- **Eventos externos**

Los primeros ocurren en tiempo de diseño y son muy fáciles de detectar porque la mayoría de los IDE analizan el cumplimiento de las convenciones y estructuras en tiempo real; en el peor de los escenarios, se haría evidente al momento de compilar el código.

Los errores de lógica, en cambio, son mucho más difíciles de detectar porque cumplen con la sintaxis y semántica, y **únicamente pueden ser descubiertos cuando el programa está en plena ejecución.**

Sin duda este es **el caso más delicado** porque dependiendo del comportamiento, **podrían generarse resultados erráticos o inesperados con diversos grados de severidad;** los bugs, como se los suele llamar, habitualmente son clasificados como triviales, importantes, muy importantes y severos.

Algunos bugs podrían pasar desapercibidos durante mucho tiempo y sin impedir el funcionamiento de la solución informática; en cambio, otros, quizá causen que el sistema deje de funcionar tras generar resultados (datos o eventos) imposibles de tratar por un proceso posterior.

Una excepción ocurre cuando el flujo normal es alterado a raíz de un evento no contemplado, pudiendo deberse a un error de lógica u otro factor externo, resultando en la terminación inmediata del programa sin el adecuado tratamiento.

La jerarquía que Javascript define para el manejo de errores provee la clase base **Error**. A partir de la misma podemos derivar clases propias para realizar un manejo de errores más efectivo.

Control de excepciones

Para lidiar con las excepciones e impedir que el sistema quede en un estado inconsistente, la mayoría de los lenguajes de alto nivel implementan **el bloque de control** conocido como **try/catch, que tiene por objetivo ejecutar un camino alternativo** en caso de que un suceso poco habitual ocurra por alguno de los motivos mencionados anteriormente.

Por ejemplo, una división entre dos números podría desatar una excepción si el divisor adopta como valor el cero, debido a que no es posible efectuar tal operación aritmética; sin el adecuado control, el sistema terminaría inmediatamente.

La estructura de un bloque try/catch se compone de tres subbloques, uno de ellos opcional pero siempre recomendable:

- **try**
Este subbloque debe encerrar el algoritmo que se espera ejecutar en condiciones normales.
- **catch**
Debe contener el camino alternativo en caso de una excepción. Tener en cuenta que en el caso de los lenguajes tipados, suele ser obligatorio definir el tipo de excepción que se desea atrapar. Por lo general suelen ser adoptadas medidas correctivas y/o que anulan los resultados de los procesos anteriores; es habitual emplear este espacio para dejar registro del error.
- **finally**
Se trata de un subbloque opcional que, de estar presente, se ejecuta siempre independientemente de que se haya producido o no una excepción; un ejemplo podría ser el cierre de una conexión a la base de datos.

Es importante tener en cuenta que cuando una excepción es capturada por un bloque **catch**, el flujo del programa continúa normalmente después del bloque **try/catch/finally**. Si se desea propagar la excepción hacia niveles superiores del programa, es posible volver a lanzarla utilizando la instrucción **throw**.

También es fundamental declarar primero los **catch** para las excepciones más específicas, dejando para lo último la más genérica (**Exception**)

```
using System;

public class Employee
{
    public string Name { get; set; }
```

```
public Employee(string name)
{
    Name = name;
}

public string GetName()
{
    return Name;
}

public class PersonManager
{
    public string GetName(Employee employee)
    {
        try
        {
            string name = employee.GetName();
            return name;
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Error getting employee name: {ex.Message}");
            return "N/A"; // Valor por defecto si ocurre un error
        }
        finally
        {
            Console.WriteLine("The finally block always executes.");
        }
    }
}
```

Ejemplo 35.1

Considerar el siguiente algoritmo.

```
class Program
{
    static void Main()
    {
        PersonManager personManager = new PersonManager();
        string name = personManager.GetName(null); // Puede ser null
        Console.WriteLine($"Nombre: {name}");
    }
}
```

Ejemplo 37.1

La ejecución del código tendrá como resultado los siguientes renglones, la ejecución del código producirá

```
Error getting employee name: Object reference not set to an instance of an object.  
The finally block always executes.  
Nombre: N/A
```

Ejemplo 37.2

El parámetro del subbloque catch permite acceder a un conjunto de atributos y funciones que brindan datos útiles; una de ellas es el **stack trace** que imprime el registro a partir del punto en el que se produjo la excepción, elemento vital para identificar el problema.

Es habitual que el stacktrace se guarde en logs para su posterior análisis.

Excepciones personalizadas

En C# es posible crear **tipos personalizados de excepciones** extendiendo la clase base **Exception**. Esto permite ajustar el manejo de errores a las necesidades específicas del sistema.

```
using System;  
  
public class EmployeeException : Exception  
{  
    public EmployeeException(string message) : base(message)  
    {  
    }  
}
```

Ejemplo 39.2

```
using System;

public class EmployeeAttributeException : Exception
{
    public EmployeeAttributeException(string message) : base(message)
    {
    }
}
```

Ejemplo 40.1

Los dos ejemplos anteriores no hacen más que extender de la clase de excepción base, llamada **Exception** e implementar uno de los constructores; se recomienda buscar en la documentación las especificaciones de la clase **Exception**.

Como toda extensión, es posible aggiornar las excepciones personalizadas con más atributos, parámetros y todo aquello que se considere necesario.

A continuación se muestra el ejemplo 39.1 modificado para adecuarlo al uso de dos excepciones; notar que la firma del método identifica cada una de las excepciones que lanza para que puedan ser identificadas adecuadamente.

El criterio para el uso de estas dos excepciones fue arbitrario con el fin de ilustrar un ejemplo.

```
public class PersonManager
{
    public string GetLastName(Employee employee)
    {
        if (employee == null)
        {
            throw new EmployeeException("Employee is null or undefined");
        }

        if (string.IsNullOrEmpty(employee.GetLastName()))
        {
            throw new EmployeeAttributeException("Employee last name is empty or contains only whitespace.");
        }

        return employee.GetLastName();
    }
}
```

Ejemplo 40.2

A raíz de los cambios realizados a la clase **PersonManager**, la función que invoque al método **GetLastname()** tendrá que optar por atrapar la excepción (A) o burbujear (B) a un nivel superior las excepciones.

```
class Program
{
    static void Main()
    {
        try
        {
            PersonManager personManager = new PersonManager();
            string lastName = personManager.GetLastName(null); // Puede lanzar
excepciones
            Console.WriteLine($"Nombre: {lastName}");
        }
        catch (EmployeeException ex)
        {
            Console.Error.WriteLine($"Employee related error: {ex.Message}");
        }
        catch (EmployeeAttributeException ex)
        {
            Console.Error.WriteLine($"Employee attribute error: {ex.Message}");
        }
        catch (Exception ex)
        {
            Console.Error.WriteLine($"Unexpected error: {ex.Message}");
        }
    }
}
```

Caso A - Ejemplo 41.1

```
public class App
{
    public void Run()
    {
        PersonManager personManager = new PersonManager();
        string lastName = personManager.GetLastName(null); // Puede lanzar
excepciones
        Console.WriteLine($"Nombre: {lastName}");
    }
}

class Program
{
    static void Main()
    {
        try
        {
```

```
        new App().Run();
    }
    catch (EmployeeException ex)
    {
        Console.Error.WriteLine($"Employee related error: {ex.Message}");
    }
    catch (EmployeeAttributeException ex)
    {
        Console.Error.WriteLine($"Employee attribute error: {ex.Message}");
    }
    catch (Exception ex)
    {
        Console.Error.WriteLine($"Unexpected error: {ex.Message}");
    }
}
```

Caso B - Ejemplo 41.2

Algo muy importante a tener en cuenta es que **de lanzarse excepciones dentro de un bloque try** resultaría en el atrapamiento inmediato por el subbloque **catch** de la misma función.